

PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
Facultad de Ingeniería
IIC2613 Inteligencia Artificial
2026-1

Tarea 3

Reinforcement Learning y Active Learning

Fecha de entrega: Martes 5 de Mayo a las 23:59 hrs

Aspectos generales

Formato y plazo de entrega

El formato de entrega son los archivos especificados a lo largo del enunciado en sus respectivos directorios (`01 - DCCLunar` y `02 - DCCdario de Señas`). El lugar de entrega es en el repositorio de la tarea, en la branch por defecto, hasta el martes 5 de mayo a las 23:59 hrs. Para crear tu repositorio, debes entrar en el enlace del anuncio de la tarea en Canvas. Por último, recuerda que los cupones de atraso son días **reales** (hábiles o no) extra.

Integridad Académica

Este curso se adhiere al Código de Honor establecido por la universidad, el cual tienes el deber de conocer como estudiante. Todo el trabajo hecho en esta tarea debe ser **totalmente individual**. La idea es que te des el tiempo de aprender estos conceptos fundamentales, tanto para el curso, como para tu formación profesional. Las dudas se deben hacer exclusivamente al cuerpo docente a través de las *issues* en GitHub.

Por otra parte, sabemos que estás utilizando material hecho por otras personas, por lo que es importante reconocerlo de la forma apropiada. Todo lo que obtengas de internet debes citarlo de forma correcta (ya sea en formato APA, ICONTEC o IEEE). Cualquier falta a la ética y/o a la integridad académica será sancionada con la reprobación del curso y los antecedentes serán entregados a la Dirección de Pregrado.

El uso de agentes inteligentes—como ChatGPT, Gemini, Copilot u otros—está permitido en las tareas siempre y cuando se indique, al inicio de la respuesta, que si se utilizaron, que se especifique de manera clara su uso (al menos el o los prompts y respuestas) y que las respuestas de estos sean modificadas sustancialmente.

Comentarios adicionales

Es fundamental que pongan énfasis en las justificaciones de sus respuestas, cuidando la redacción, ortografía; manteniendo el código ordenado y comentado. Aquellas respuestas que solo presenten resultados o código (sin contexto ni comentarios) no serán consideradas, mientras que tareas desordenadas pueden ser objeto de descuentos.

1. DCCLunar: Aprendizaje Reforzado (3 puntos)

Has sido seleccionado por la NASA como parte del equipo encargado de entrenar una inteligencia artificial que será utilizada para asistir en el aterrizaje del módulo lunar de la misión Artemis IV, programada para el año 2028.

Tu tarea consiste en diseñar y entrenar un agente inteligente capaz de aterrizar exitosamente un módulo en un entorno simulado, utilizando técnicas de *Reinforcement Learning*. Este agente deberá aprender desde cero, únicamente a partir de la interacción con el entorno.

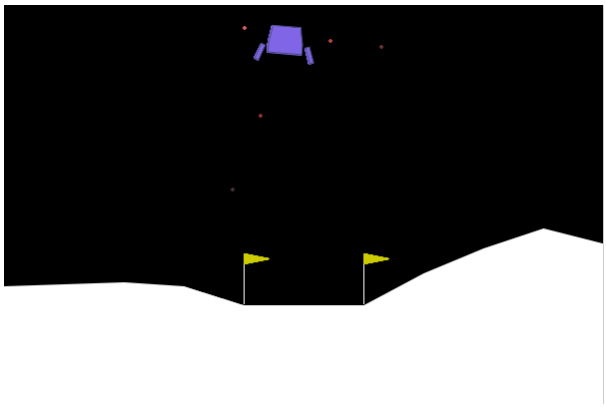


Figura 1: Simulación del entorno LunarLander.

En tu repositorio se te entrega el código base del entorno, junto con un agente inicial que actúa de manera aleatoria. Deberás implementar el algoritmo *Q-Learning* para que el agente aprenda a aterrizar de forma segura y eficiente.

La interfaz utilizada para entrenamiento no es la misma que la de visualización. Durante el entrenamiento, el agente interactúa directamente con el entorno sin renderizado para acelerar el proceso.

Los archivos contenidos en el código base (dentro del directorio `01 - DCCLunar`) son:

- `main.py`: Archivo principal de ejecución. **No modificar.**
- `train.py`: Contiene el loop de entrenamiento. Modificar
- `test.py`: Permite evaluar el agente entrenado. **No modificar.**
- `LunarLanderAgent.py`: Contiene la clase del agente que se mueve de manera aleatoria. Es aquí donde deberás trabajar. Modificar

Como ya se mencionó, el archivo sobre el cual debes trabajar es `LunarLanderAgent.py`[1]. Este archivo contiene la clase `LunarLanderAgent` con los siguientes métodos:

- `__init__()`: Inicializa los atributos del agente. En el caso del agente aleatorio, solo se inicializan en 0 las partidas jugadas por el agente.
- `get_state(observation)`: Consulta por el estado actual del agente y lo retorna como una tupla de 7 dimensiones discretas [2], donde cada entrada representa (en orden):

[1] Este archivo se encuentra comentado, por lo que se recomienda leerlo como parte del enunciado

[2] Cada variable continua del entorno es discretizada en 5 categorías. El valor entero retornado no representa la magnitud de la variable, sino el intervalo al cual pertenece.

1. **Posición horizontal (x_d)**: Toma un valor entero entre 0 y 4 que corresponde a la coordenada en el eje X de la nave.
2. **Posición vertical (y_d)**: Toma un valor entero entre 0 y 4 que corresponde a la coordenada en el eje Y de la nave.
3. **Velocidad horizontal ($v_{x,d}$)**: Toma un valor entero entre 0 y 4 que corresponde a la velocidad linear de la nave en el eje X.
4. **Velocidad vertical ($v_{y,d}$)**: Toma un valor entero entre 0 y 4 que corresponde a la velocidad linear de la nave en el eje Y.
5. **Ángulo (θ_d)**: Toma un valor entre 0 y 4 que corresponde al ángulo de inclinación actual de la nave.
6. **Velocidad angular (ω_d)**: Toma un valor entre 0 y 4 que corresponde a la velocidad angular actual de la nave.
7. **Estado de contacto de patas (c)**: Es un entero que puede tomar los valores 0, 1 2 y 3. Toma el valor 0 si ninguna pata se encuentra en contacto con el piso. Toma el valor 1 si solo la pata derecha está en contacto, y el valor 2 si solo la pata izquierda está en contacto. Si ambas se encuentran en contacto, toma el valor 3.

- `get_action(state)`: Recibe el estado del agente y retorna un entero representando la acción que debe seleccionar el agente. Las acciones posibles son:
  - `0`: No hacer nada.
  - `1`: Activar motor izquierdo.
  - `2`: Activar motor principal (central).
  - `3`: Activar motor derecho.

  Nota: En el caso del agente aleatorio, este no utiliza la información que entrega el estado y retorna una acción al azar.
- `update_q_value(state, action, reward, next_state, done)`: Actualiza la *Q-Table* para una tupla (estado, acción) usando *Q-Learning*.
- `decay_exploration_rate()`: Actualiza la tasa de exploración una vez por episodio.

Actividad 1: Comprendiendo los hiperparámetros (0.5 puntos)

Caso: Dos estudiantes entrenan agentes con configuraciones diferentes. El Agente A usa learning rate $\alpha_A = 0.1$ y discount factor $\gamma_A = 0.9$, mientras que el Agente B usa $\alpha_B = 0.8$ y $\gamma_B = 0.1$. Después de 10000 episodios, el Agente A muestra mejoras graduales y consistentes, pero el Agente B tiene oscilaciones

Nota: Para esta actividad no es necesario tocar el código.

1. Investiga la ecuación de actualización de Q-Learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \operatorname{máx} Q(s',a') - Q(s,a)]$$

   ¿Cómo explica matemáticamente esta ecuación las diferencias entre ambos agentes? Analiza el impacto específico de α_A y α_B en la magnitud de las actualizaciones.

2. Investiga cómo funciona la exploración ϵ-greedy y toma en cuenta la siguiente fórmula para el decaimiento de ϵ:

   $\epsilon = \epsilon_{inicial} * decay^{n_de_episodios}$

   Si ambos agentes comienzan con $\epsilon = 1.0$ y decay = 0.995, ¿en qué episodio aproximadamente cada uno tendría ϵ =0.1? ¿Cómo interactúa esto con sus diferentes learning rates?

   Bonus por 0.2 puntos: Investiga sobre el decaimiento de ϵ ¿existen más además del dado en la fórmula anterior?

3. ¿Por qué el Agente B experimenta oscilaciones mientras que el Agente A mejora gradualmente? ¿Qué hiperparámetro específico causa esta inestabilidad y cómo?

4. El agente A con $\gamma_A = 0.9$ planifica más hacia el futuro que el Agente B con $\gamma_B = 0.1$. En el contexto de LunarLander, ¿qué ventajas específicas tendría el Agente A al anticipar la trayectoria de su nave?

5. Si quisieras combinar lo mejor de ambos agentes (la velocidad inicial del B y la estabilidad del A), ¿qué configuración de hiperparámetros propondrías y por qué? Justifica cómo cada parámetro contribuiría al objetivo deseado.

Actividad 2: Implementando Q-Learning (0.75 puntos)

Básandote en lo visto en clases, implementa el algoritmo *Q-Learning*[3] para el agente de la nave. Para esto, deberás realizar los siguientes pasos en `LunarLanderAgent.py`[4]:

1. Inicializar la *Q-Table* en el método `__init__()` con sus dimensiones correctas **(0.125 pts)**.

2. Modificar el método `get_action()` para que el agente sea capaz de explorar el estado actual o explotar la mejor acción conocida hasta el momento **(0.25 pts)**.

3. Modificar el método `update_q_value()` para actualizar la *Q-Table* una vez que se ejecute la acción seleccionada por el agente **(0.2 pts)**. La función debe:

   a) Buscar los índices de los estados en la tabla de estados. **No modificar**.

   b) Calcular el mejor valor Q del siguiente estado (o 0 si el episodio terminó).

   c) Calcular el objetivo temporal (`td_target`) sumando la recompensa y el descuento del mejor siguiente Q.

   d) Calcular el error temporal (`td_error`) restando el valor Q actual del objetivo.

   e) Ajusta el valor Q actual en la tabla usando la tasa de aprendizaje `LR`.

   f) Guardar el error de entrenamiento para seguimiento. **No modificar**.

4. Modificar el método `decay_exploration_rate()` para, al terminar una partida (episodio), actualizar el *exploration_ rate* del agente **(0.175 pts)**.

IMPORTANTE Y OBLIGATORIO

Para facilitar la corrección de tu tarea, debes subir la *Q-Table* de tu agente ya entrenado como un archivo en formato `<usuario_github>.csv` (es decir, tu usuario de GitHub), sin *headers* (esto es, filas con títulos). Este debe estar dentro del directorio `RL/`.

El archivo `.csv` debe poseer 11 columnas y 62500 filas. Las primeras 7 columnas corresponden a los valores de las 7 dimensiones de los estados en el orden descrito en este enunciado. Por otra parte, las siguientes 4

[3] Si tienes dudas con *Q-Learning*, puedes revisar este artículo.

[4] Se recomienda revisar los `#TODO` que hay en el código.

columnas corresponden al *Q-Value* de las acciones posibles del agente en el orden del enunciado. De esta forma, cada fila representa un posible estado y los valores de las acciones posibles para este estado. Una fila de este archivo se debe ver así:

$$[x_d,\ \ y_d,\ \ v_{x,d},\ \ v_{y,d},\ \ \theta_d,\ \ \omega_d,\ \ c,\ \ Q_0,\ \ Q_1,\ \ Q_2,\ \ Q_3]$$

Este formato también es el requerido para poder participar en el bonus, así que estás invitado/a a participar.

Actividad 3: Experimentación y Análisis del Agente (1.5 puntos)

Para esta sección, deberás realizar cuatro experimentos distintos modificando los hiperparámetros del agente. Para cada experimento, debes **generar evidencia** en el archivo `train.py` y usarla para fundamentar tus respuestas.

- **Experimento 1: El Agente "Miope"vs. el Agente "Previsor"(0.375 puntos)**

  El código inicial utiliza un `DISCOUNT_RATE = 0`, lo que crea un agente "miope"(solo le importa la recompensa inmediata). Para este experimento, deberás seguir los siguientes pasos:

  1. Realiza dos entrenamientos: uno con el valor original ($\gamma = 0$) y otro con un valor alto que incentive al agente a ser "previsor"(por ej., $\gamma = 0.95$). Entrena a cada uno por un número significativo de episodios (ej. 30.000).
  2. Genera un gráfico que muestre el `mean_score` (puntaje promedio) para **cada entrenamiento** en la misma figura.
  3. Analiza el gráfico, responde: ¿Qué agente aprende más rápido al principio? ¿Cuál alcanza un mejor rendimiento máximo a largo plazo? ¿Cómo la evidencia visual de tu gráfico demuestra la diferencia fundamental entre un agente que busca recompensas inmediatas y uno que planifica para el futuro? En base a esto, ¿qué valor de γ elegirías?

- **Experimento 2: El Dilema de la Velocidad vs. la Estabilidad (0.375 puntos)**

  El `LR` (Learning Rate) no solo afecta qué tan rápido aprende el agente, sino que también qué tan estable es su conocimiento. Para este experimento, deberás seguir los siguientes pasos:

  1. Realiza un experimento comparando un `LR` muy bajo (ej. `0.01`) contra un `LR` muy alto (ej. `0.7`).
  2. Para cada caso, registra el `record` (puntaje máximo) y el `mean_score` cada 100 episodios. Presenta tus resultados en una tabla comparativa.
  3. Basado en la tabla, compara el proceso de aprendizaje del agente con tasas de aprendizaje (`LR`) bajas y altas. Describe la velocidad y las características de su rendimiento (p. ej., estable, errático, estancamiento) en cada escenario. Finalmente, selecciona el valor de `LR` que consideres óptimo y justifica tu elección basándote en el compromiso (*trade-off*) observado entre la velocidad de convergencia y la estabilidad del rendimiento.

- **Experimento 3: Saboteando el Aprendizaje (0.375 puntos)**

  El balance entre explorar el entorno y explotar el conocimiento adquirido es fundamental. En este experimento, investigaremos qué pasa cuando ese balance se rompe. Para ello, deberás seguir los siguientes pasos:

  1. Diseña y ejecuta dos experimentos "fallidos":
     a) **Exploración Prematura**: Modifica el `EXPLORATION_DECAY_RATE` a un valor muy alto (ej. `0.05`) para que el agente deje de explorar casi inmediatamente.

b) **Exploración Eterna**: Modifica el `MIN_EXPLORATION_RATE` para que sea igual a `MAX_EXPLORATION_RATE` (ej. `0.8`), forzando al agente a tomar una acción aleatoria el 80 % de las veces, para siempre.

2. Describe el comportamiento observado en los escenarios de "Exploración Prematuraz "Exploración Eterna". Evalúa la calidad de las políticas aprendidas y fundamenta cómo el balance entre exploración y explotación en cada caso explica el rendimiento final del agente.
3. Explica cómo los resultados de estos dos experimentos demuestran por qué es crucial tener una tasa de exploración que **decaiga gradualmente** con el tiempo.

- **Experimento 4: Entrenando a tu Agente Ideal (0.375 puntos)**

  Hemos descubierto que los hiperparámetros del agente dictan su desempeño (ya sea para bien o para mal). Para este último experimento, diseñaremos y entrenaremos a nuestro "agente ideal". Para ello, deberás seguir los siguientes pasos:

  1. Selecciona y justifica brevemente la configuración de hiperparámetros que consideres óptima y reentrena al agente con ella. Recomendamos usar al menos `30_000` simulaciones.
  2. **Genera un gráfico** que muestre el `mean_score` (puntaje promedio) a lo largo del tiempo del entrenamiento.
  3. Reflexiona sobre el rendimiento de tu agente. Considera si un agente "perfecto."[es] un objetivo realista y describe qué acciones específicas tomarías para mejorar aún más los resultados obtenidos, justificando tu propuesta.

Actividad 4: Política de recompensas (0.25 puntos)

Actualmente, el juego otorga recompensas al agente en cada paso según el desempeño del aterrizaje[5]. Estas aumentan cuando el módulo se acerca a la zona de aterrizaje, se mueve lentamente y logra contacto con el suelo; y disminuyen cuando se aleja del objetivo, se inclina o utiliza sus motores. Además, el episodio recibe una gran recompensa si el aterrizaje es exitoso o un castigo si la nave se estrella.

Para esta actividad deberás analizar la política de recompensas que usa el juego actualmente para poder entrenar al agente y responder:

- ¿Por qué crees que es (o no es) una buena forma de recompensar al agente? Argumenta.
- ¿Qué cambios o modificaciones harías a la manera de recompensar al agente?

Bonus: DCCampeonato IIC2613 2026-1

Celebraremos la **tercera edición** del DCCampeonato IIC2613, en donde pueden poner a prueba sus agentes de aprendizaje reforzado y competir con sus demás compañeros de curso en un torneo de aterrizaje. Los primeros 5 lugares **podrán ganar hasta 10 décimas** en su tarea, además de entrar al salón de la fama del curso.

[5] Si quieres leer el sistema de recompensas específico, puedes revisar esta documentación.



El entregable para participar de este campeonato es:

1. La *Q-Table* del agente entrenado, en el mismo formato requerido para la entrega y corrección de la tarea (debe llamarse `<usuario_github>.csv`).

¿Qué entregar en esta parte?

Se deberá entregar los archivos `train.py` y `LunarLanderAgent.py` modificados, un archivo `respuesta.pdf` con tus respuestas y la *Q-Table* del agente entrenado, todo dentro del directorio `01 - DCCLunar`.

2. DCCdario de Señas (3 puntos)



2.1. Introducción

Es tu primer día de clases y llegas con mucho entusiasmo al campus con el objetivo de ubicar tus primeras sala de clases de este semestre, la A7, BC24 y la ED219 pero por más que buscas no sabes dónde se encuentran estas salas. Abrumado por la cantidad de salas y el tamaño del campus, decides pedir indicaciones, pero la persona que te ayuda se comunica en lengua de señas, por lo que no logras entender y decides retirarte derrotado a tu casa. Te propones no volver a pasar por esto y decides construir un modelo de Machine Learning capaz de traducir el Lenguaje de Señas Americano (ASL) a texto utilizando solo una cámara web (podría ser más fácil buscar las salas, pero tú ya estás decidido a no perder de nuevo).

Logras recopilar un dataset crudo gigante con miles de fotografías de manos realizando las 5 letras que necesitas. Sin embargo, rápidamente te estrellas contra un gran muro en el mundo de la Inteligencia Artificial: las imágenes no tienen etiquetas.

Para que tu clasificador supervisado aprenda, alguien tiene que revisar foto por foto e indicar meticulosamente si esa mano está haciendo una A, una B o una C. Contratar a un experto en lengua de señas cuesta dinero que no tienes, y si decides hacerlo tú mismo, te tomará semanas de trabajo repetitivo. Tus recursos; tiempo, dinero y sobre todo paciencia, son estrictamente limitados y no puedes etiquetar más de 150 fotos. Te das cuenta de que etiquetar fotos al azar es ineficiente porque muchas imágenes son redundantes o fáciles de clasificar.

Justo cuando estás considerando abandonar el proyecto y dejar de ir a la universidad, recuerdas una de tus clases de Inteligencia Artificial en el que el profesor habló de una estrategia diseñada exactamente para este tipo de crisis de recursos: el **Active Learning (Aprendizaje Activo)**. Con esta estrategia decides darle la vuelta al problema, en lugar de alimentar al algoritmo pasivamente, le darás el control: dejarás que el modelo analice todo tu pool de fotos sin etiquetar, calcule matemáticamente sus propias dudas y te pida ayuda (como si fueras un "Oráculo") exclusivamente con las imágenes que más lo confunden.

2.1.1. Objetivo general

El objetivo general de esta sección de la tarea es desarrollar modelos de aprendizaje automático capaces de reconocer gestos de la mano —específicamente un subconjunto del Alfabeto de Lengua de Señas Americano (ASL), correspondiente a las letras *A, B, C, D* y *E*— a partir de imágenes estáticas.

Para lograrlo, se utilizarán los *landmarks* (o coordenadas) resultantes de la librería MediaPipe[6], los cuales describen la posición tridimensional de 21 puntos característicos de la mano. A partir de estos datos, se entrenarán y evaluarán distintos modelos de clasificación supervisada.

[6] https://ai.google.dev/edge/mediapipe/solutions/guide

Además, esta tarea introduce el paradigma de ***Active Learning*** (Aprendizaje Activo). En lugar de entrenar modelos con miles de datos pre-etiquetados, simularás entornos donde el etiquetado es costoso. Tomarás el rol de un "Oráculo" humano que interactúa con el modelo, permitiendo que el algoritmo seleccione de manera inteligente qué imágenes necesita que le etiquetes para maximizar su aprendizaje con el menor esfuerzo posible.

Se espera que seas capaz de comprender el flujo completo, desde la extracción de *landmarks* y preprocesamiento, hasta la implementación de estrategias de muestreo y la evaluación dinámica del desempeño. Algunas de las funciones de esta librería **requieren del uso de una GPU**, por lo que te recomendamos ajustar el entorno de ejecución desde **Google Colab**.

2.1.2. Dataset

El dataset utilizado en esta tarea corresponde al *ASL Alphabet Dataset*[7] disponible en la plataforma *Kaggle*.

Para desarrollar correctamente esta actividad, se te entrega un archivo base dentro del directorio `02 - DCCdario de Señas` llamado `Traductor_AL.ipynb` (Jupyter notebook) que contiene el código que descarga y procesa las imágenes `RGB` del *dataset* mediante MediaPipe. Este código extrae los datos y genera el conjunto `train_landmarks.csv` que contiene los siguientes datos:

- **Filename**: Corresponde al nombre original de la imagen procesada.
- **Label**: Corresponde a la categoría de la foto (en este caso, las letras *A, B, C, D* o *E*).
- $x_i, \forall i \in \{0, 1, \ldots, 62\}$: corresponden a los *landmarks* generados por `mediapipe`. Puesto que la mano contiene 21 *landmarks*, y cada *landmark* tiene 3 coordenadas (X,Y,Z), se obtienen $21 \cdot 3 = 63$ características espaciales por imagen.

2.2. Actividades

2.2.1. Parte 1: Creación de características

(1 punto)

A partir de los *landmarks* extraídos (las 63 coordenadas X, Y, Z de la mano), deberás fabricar al menos **5 *features* (características) nuevas** para entrenar tus modelos de *machine learning*. Considera que un *landmark* o punto de referencia es un punto de interés clave y localizable, en nuestro contexto, describen la anatomía de la mano con coordenadas en el espacio (X,Y,Z) normalizadas respecto al tamaño de la imagen.

Podemos pensar en la mano como un grafo no dirigido $G = (V, E)$, donde:

- Los nodos V corresponden a los *landmarks* (los 21 puntos de la mano).
- Las aristas E corresponden a las tuplas definidas en `hand_connections` que indican qué puntos están conectados biomecánicamente.

Formalmente:

$$V = \{0, 1, 2, \ldots, 20\}$$

$$E = \{(0, 1), (1, 2), (2, 3), \ldots, (19, 20), (0, 17)\}$$

Esquemáticamente:

[7] https://www.kaggle.com/datasets/grassknoted/asl-alphabet

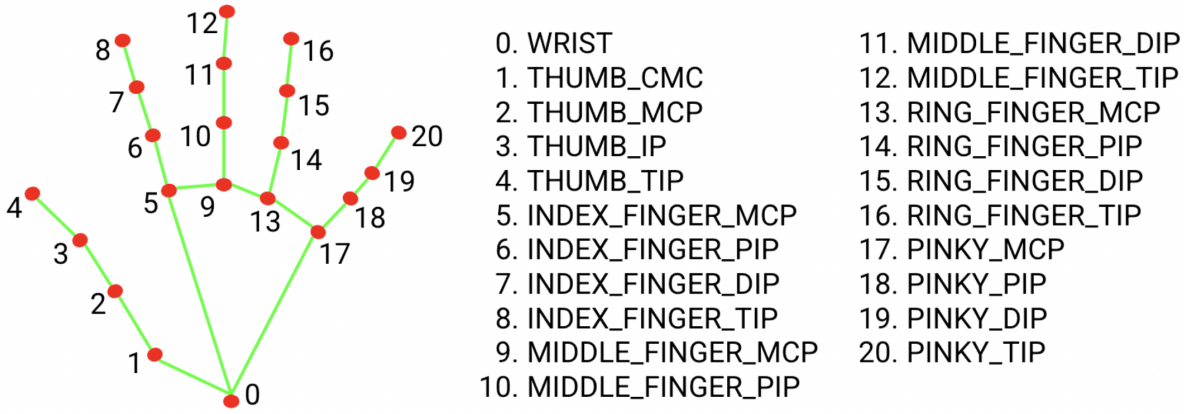


Figura 2: Modelado general de la mano

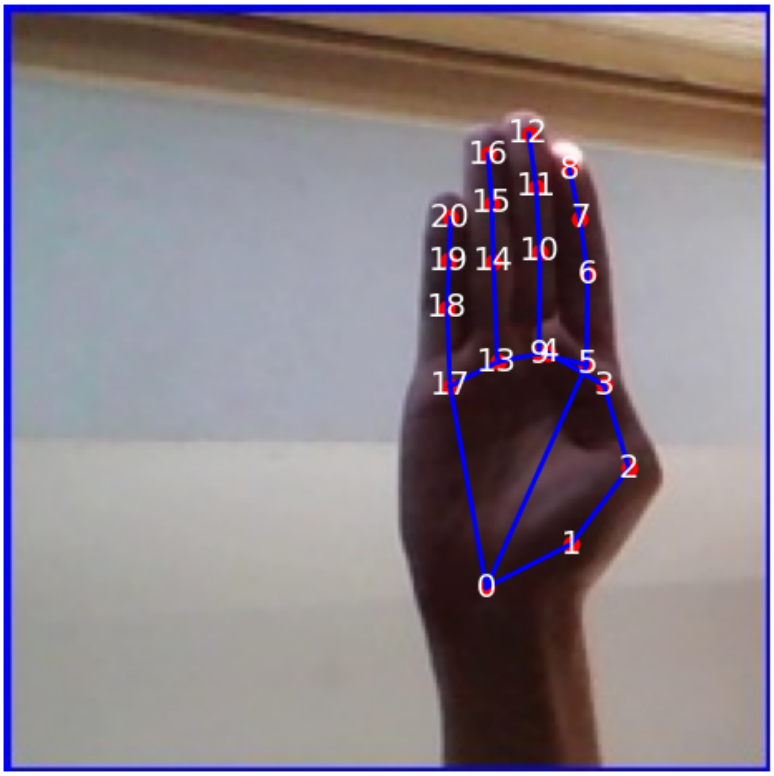


Figura 3: Output con landmarks de la imagen B/B300.jpg ubicada en la carpeta asl_alphabet_train.

Preprocesamiento (*Scaling*): Es obligatorio que normalices tu *dataset* (por ejemplo, con `StandardScaler`) antes de entrenar. Esto es vital porque algoritmos como el *SVM* son extremadamente sensibles a la escala; al basarse en distancias geométricas, si no normalizas, las *features* con valores numéricos grandes dominarán artificialmente las decisiones del modelo.

Para cada una de las 5 características creadas:

- Explica brevemente qué hace o a qué corresponde geométricamente en la mano.
- Justifica brevemente su uso y explica por qué crees que será útil para obtener un buen desempeño diferenciando específicamente las letras *A, B, C, D* y *E*.

(**Nota: Está prohibido usar los *landmarks* crudos como únicas *features*. Debes crear tus propias características en base a ellos. En caso contrario, no será corregida esta parte de la tarea**).

2.2.2. Parte 2: Active Learning - Simulación Manual

(1 punto)

En esta sección te convertirás en el "Oráculo". Deberás instanciar 3 clasificadores base (*Decision Tree, Random Forest* y *SVM*) y definir 4 estrategias de muestreo utilizando la librería `skactiveml`: una estrategia aleatoria (*RandomSampling*) como línea base, dos estrategias basadas en incertidumbre (*Entropy* y *Margin Sampling*) y una estrategia de *Query by Committee* (QBC).

Para facilitar tu trabajo con la interfaz gráfica y que puedas enfocarte exclusivamente en la lógica del Machine Learning, se te proporcionarán dos funciones auxiliares listas para usar:

- `registrar_metricas_viewer`: Calcula automáticamente las métricas de desempeño (*Accuracy, Precision, Recall*) en la iteración actual y actualiza los gráficos del *Viewer*.
- `interactuar_con_oraculo`: Despliega la interfaz visual para que tú (el Oráculo) etiquetes las imágenes que seleccionó el algoritmo, encargándose de actualizar internamente el *pool* de datos con tus respuestas.

Tu misión será implementar **desde cero** la función principal `run_manual_active_learning`. Deberás orquestar el ciclo completo: preparar los datos, entrenar el modelo iterativamente y utilizar la estrategia elegida para determinar qué nuevas imágenes del *pool* deben ser reveladas (haciendo uso de las funciones auxiliares en los momentos adecuados). Al lograrlo y ejecutar la celda, experimentarás de primera mano cómo el algoritmo te pide ayuda estratégicamente.

¡Atención con Query by Committee (QBC)! A diferencia de las estrategias de incertidumbre que evalúan las probabilidades de un solo clasificador, QBC requiere matemáticamente de un "comité" que debata y busque las muestras con mayor nivel de desacuerdo. Esto implica un paso algorítmico adicional en tu implementación para los modelos individuales como *Decision Tree* o *SVM* que a diferencia de *Random Forest* no son naturalmente ensambles.

Una vez implementada la función de manera robusta, deberás ponerla a prueba interactuando con la interfaz gráfica. Para completar esta sección, deberás incluir lo siguiente:

- **Evidencia visual y de métricas:** Adjunta una tabla resumen o los gráficos finales generados por la interfaz para al menos 8 de las 12 combinaciones de modelos y estrategias, siendo **obligatorio** realizar 2 combinaciones por cada una de las 4 estrategias. (Asegúrate de mostrar el *Accuracy, Precision* y *Recall* final alcanzado en cada iteración).
- **Análisis comparativo:** Compara los resultados de tus simulaciones. ¿Qué diferencias observas en la velocidad de aprendizaje al utilizar una selección aleatoria versus una basada en incertidumbre o en el desacuerdo del comité? ¿Hubo algún modelo (DT, RF, SVM) que se beneficiara más del *Active Learning* que los otros?
- **La experiencia del Oráculo:** Basándote en tu interacción directa con la interfaz, ¿notaste visualmente algún patrón en las imágenes de las manos que el modelo te pedía etiquetar cuando usabas *Uncertainty Sampling* o *QBC*? ¿Te parecieron casos genuinamente más difíciles, ambiguos o inusuales en comparación con el muestreo aleatorio? Justifica tu respuesta conectando tu experiencia visual con el funcionamiento algorítmico.

2.2.3. Parte 3: Simulación Automatizada de Active Learning

(1 punto)

En esta etapa, dejaremos de etiquetar manualmente y automatizaremos el rol del "Oráculo" extrayendo las etiquetas reales del *dataset*. Esto nos permitirá realizar una simulación a mayor escala para evaluar el comportamiento de nuestros 3 modelos combinados con las diferentes estrategias de muestreo.

A diferencia de la parte anterior, en esta sección **no se te entregará código base**. Deberás implementar desde cero tu propio ciclo de *Active Learning* automatizado respetando estrictamente las siguientes reglas de juego:

- **Arranque en frío (*Cold Start*):** Para que el algoritmo pueda hacer su primera predicción, deberá comenzar con 20 muestras iniciales "gratuitas" (asegúrate de que esta selección inicial sea estratificada para que el modelo conozca al menos 4 instancias de cada clase antes de arrancar).
- **Presupuesto de Tokens:** Cuentas con un límite estricto de **150 Tokens**. Cada token equivale a una nueva imagen que tu código le solicitará automáticamente al Oráculo para etiquetar. Tu ciclo debe detenerse inmediatamente cuando se agote este presupuesto.
- **Gestión del *Batch Size*:** Tú deberás definir y programar cómo se gestionan las consultas en cada iteración mediante el tamaño del lote (*Batch Size*). Deberás experimentar para descubrir qué es más conveniente: ¿pedir lotes pequeños (más iteraciones, permitiendo que el modelo se corrija paso a paso) o lotes grandes (menos iteraciones, etiquetando imágenes más rápido pero tal vez gastando tokens en datos redundantes)?

Al finalizar la simulación, deberás incluir lo siguiente:

- **Gráficos de evolución:** Genera una figura con subgráficos (o una gráfica unificada clara) que compare la evolución del *Accuracy, Precision Macro* y *Recall Macro* a medida que se consumen los tokens para tus simulaciones con distintos *Batch Size*.
- **Análisis de Estrategias y Modelos:** ¿Qué combinación de modelo y estrategia permitió alcanzar un mejor desempeño gastando la menor cantidad de tokens? ¿Existen diferencias notables entre cómo aprovecha la incertidumbre un modelo simple (como el Árbol) versus un ensamble (como *Random Forest*)?
- **El impacto del *Batch Size*:** Basado en tus experimentos variando el tamaño del lote, ¿existe un *trade-off* (compensación) entre el tamaño del *batch* y la precisión final alcanzada por los modelos? ¿Cuál consideras que es el punto de equilibrio empírico entre la cantidad de iteraciones y las fotos por iteración para este problema específico?

2.2.4. Bonus: El Oráculo en el Mundo Real

(0.5 puntos)

Es hora de poner a prueba todo lo desarrollado y evaluar si el mejor modelo de tu simulación logró generalizar correctamente en un entorno real, fuera del *dataset* original.

1. **Captura:** Tómate una fotografía con tu cámara realizando una de las letras permitidas (*A, B, C, D* o *E*) en Lenguaje de Señas Americano (ASL).
2. **Extracción de *Features*:** Pasa tu fotografía por la función de extracción de la Parte 1.
3. **Preprocesamiento:** Si durante tu entrenamiento utilizaste algún escalador (como `StandardScaler` o `MinMaxScaler`), aplícalo a tu nuevo vector de datos.

4. **Predicción:** Utiliza el modelo y la estrategia ganadora de tu simulación automatizada (Parte 3) para predecir qué letra estás haciendo.

5. **Visualización:** Muestra tu fotografía con los *landmarks* de tu mano superpuestos y con el resultado de la predicción.

¿Qué entregar en esta parte?

Se deberá entregar el archivo base `Traductor_AL.ipynb` modificado según las actividades pedidas **con todas las celdas ejecutadas; de lo contrario no será corregido**. Dicho notebook deberá poder funcionar al momento de la corrección para obtener el puntaje.

Importante: No debes subir los datos ni las imágenes descargadas a tu repositorio de GitHub, de lo contrario habrá una penalización importante en tu nota.